Simulating OU Dynamics

Implementation of the Forward Process

The Ornstein-Uhlenbeck Process

$$dx = -\frac{1}{2}\beta(t)x\,dt + \sqrt{\beta(t)}\,dW_t \tag{1}$$

Numerical Discretization: Euler-Maruyama

Discretizing into finite time steps Δt, the update rule becomes:

$$x_{i+1} = x_i - \frac{1}{2}\beta(t_i)x_i\Delta t + \sqrt{\beta(t_i)\Delta t}\,\epsilon_i \tag{2}$$

where $\epsilon_i \sim \mathcal{N}(0,1)$.

Simulation of the 1D OU Forward Process

Constant Temperature Case ($\beta(t) = \beta_0 = 1$)

- **Analytic Mean:** $\mu(t) = x_0 \exp\left(-\frac{1}{2}\beta_0 t\right)$
- **Analytic Variance:** $\sigma^2(t) = 1 - \exp(-\beta_0 t)$

As $t \to \infty$, the distribution converges to the stationary state $p_\infty(x) = \mathcal{N}(0, 1)$.

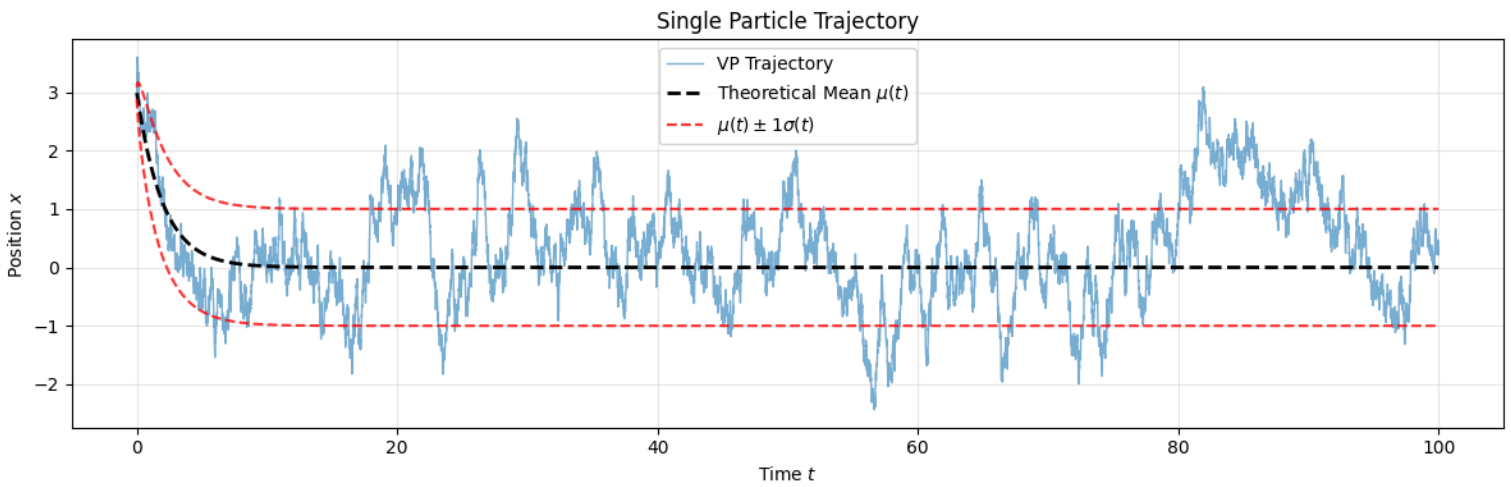


Figure: Relaxation from $x_0 = 3.0$

Ensemble Evolution of the Forward Process

Simulating an ensemble of particles drawn from

$$p_{\text{initial}}(x, 0) = \frac{1}{2}\mathcal{N}(-2, 0.3^2) + \frac{1}{2}\mathcal{N}(2, 0.3^2) \tag{3}$$

which leads to convergence to the stationary distribution:

$$p_{\text{final}}(x, T) = \mathcal{N}(0, 1) \tag{4}$$

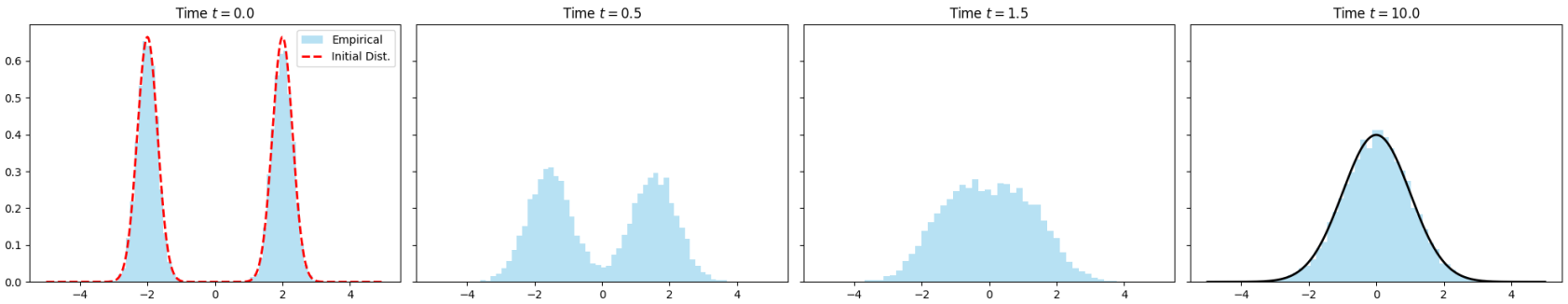


Figure: Snapshots of the 1D density evolution using $10,000$ particles.

KL Divergence at T=499: 0.006790

Directly Sampling the Distribution

Using the analytic results

- **Gaussian Distribution**
- **Analytic Mean:** $\mu(t) = x_0 \exp\left(-\frac{1}{2}\beta_0 t\right)$
- **Analytic Variance:** $\sigma^2(t) = 1 - \exp(-\beta_0 t)$

one can do a reparameterization

$$x_t = x_0 e^{-\frac{1}{2}\beta_0 t} + \sqrt{1 - e^{-\beta_0 t}}\epsilon, \quad \epsilon \sim \mathcal{N}(0, 1) \tag{5}$$

Sample any state x_t in a single step.

Direct Sampling with Discrete Schedule

If there is a temperature schedule β_t

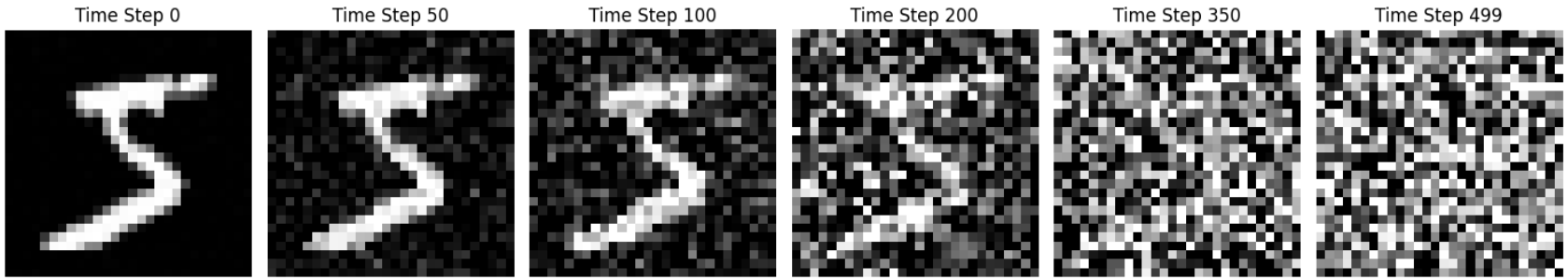
- **Analytic Mean:** $\mu(t) = x_0 \exp(-\frac{1}{2}\sum_{i=1}^{t} \beta_i \Delta t)$
- **Analytic Variance:** $\sigma^2(t) = 1 - \exp(-\sum_{i=1}^{t} \beta_i \Delta t)$

Using $\bar{\alpha}_t = \prod_{i=1}^{t} \exp(-\beta_i \Delta t) \approx \prod_{i=1}^{t}(1-\beta_i)$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0,1) \tag{6}$$

Sample any state x_t in a single step.

Diffusion of a MNIST Image



- 28^2-d forward diffusion of a MNIST digit
- $T = 500$ steps
- linear schedule $\beta_0 = 0.0001$ to $\beta_T = 0.02$
- using direct sampling

The Backward Process

Anderson's Theorem (1982)

Any forward SDE $dx = f(x,t)dt + g(t)dW$ has a backward SDE:

$$dx = \left[f(x,t) - g(t)^2 \nabla_x \log p_t(x)\right] dt + g(t)dW \tag{7}$$

where $\nabla_x \log p_t(x)$ is the **Score Function**.

The Score Function is just the Gaussian Noise

Using $p(t) = q(x_t|x_0) = \mathcal{N}(\mu(t), \sigma^2(t))$

$$\nabla_{x_t} \log q(x_t|x_0) = \nabla_{x_t} \left[-\frac{(x_t - \mu(t))^2}{2\sigma^2(t)}\right] = -\frac{x_t - \mu(t)}{\sigma^2(t)} \tag{8}$$

Substitute $x_t = \mu(t) + \sigma(t)\epsilon$:

$$\nabla_{x_t} \log q(x_t|x_0) = -\frac{\sigma(t)\epsilon}{\sigma^2(t)} = -\frac{\epsilon}{\sigma(t)} \tag{9}$$

Learning the Score and Generating Data

Estimating the Score Function

- A neural network $\epsilon_\theta(x_t, t)$ is trained to predict the noise ϵ that was used to generate x_t from x_0 at time t.

Solving the Reverse SDE

- The score is estimated as $\nabla_x \log p_t(x) \approx -\frac{\epsilon_\theta(x_t,t)}{\sigma(t)}$.
- Then the backward SDE can be discretized as:

$$x_{t-\Delta t} = x_t - \left[-\frac{1}{2}\beta x_t + \frac{\beta}{\sigma(t)}\epsilon_\theta(x_t, t)\right]\Delta t + \sqrt{\beta \Delta t}\,\epsilon$$

where $\epsilon \sim \mathcal{N}(0, 1)$.

Simulating the Reverse Process

```python
class ScoreNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 12), nn.ReLU(),
            nn.Linear(12, 12), nn.ReLU(),

            nn.Linear(12, 1)
        )

    def forward(self, x, t):
        inp = torch.cat([x, t], dim=1)
        return self.net(inp)
```

Figure: Simple MLP: Predict the noise from x_t and t

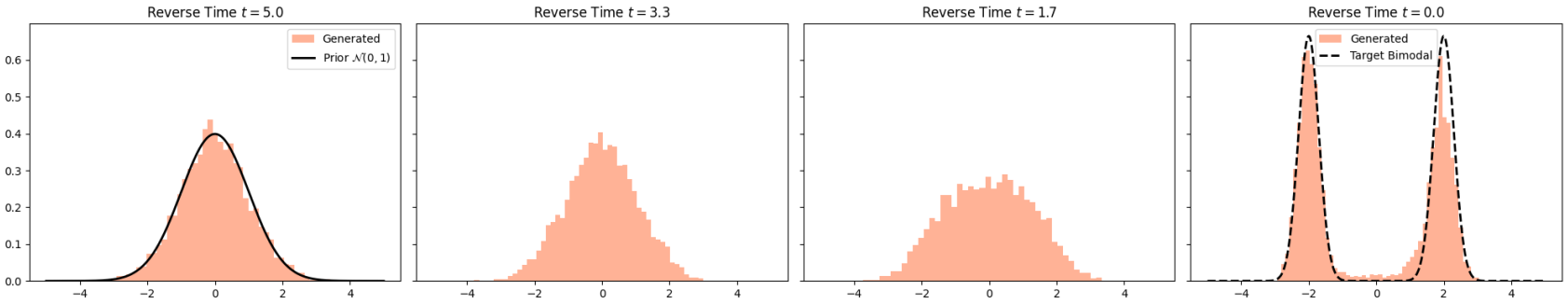


Figure: The bimodal density can be recovered.